



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ _____ Фундаментальные науки

КАФЕДРА _____ Прикладная математика

Отчет по лабораторной работе №2 по курсу "Методы оптимизации"

Градиентный спуск

Студент _____
ФН2-61Б
(Группа)

(Подпись, дата)

И. Е. Дыбко

(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

А. В. Чередниченко

(И. О. Фамилия)

2025 г.

Оглавление

1. Постановка задачи	3
2. Листинг	3
3. Визуализация работы алгоритмов	6
4. Результаты работы алгоритмов	13
5. Выводы	15

1. Постановка задачи

Рассмотрим задачу поиска наименьшего значения скалярной функции $f(\mathbf{x})$ на множестве её области определения $D(f)$:

$$f(\mathbf{x}) \rightarrow \min, \quad \mathbf{x} \in D(f) \subset \mathbb{R}^n.$$

Требуется найти значение

$$f_* = f(\mathbf{x}_*)$$

и точку $\mathbf{x}_*$, в которой функция $f(\mathbf{x})$ достигает этого значения. Точку $\mathbf{x}_*$ будем искать с точностью 10^{-2} и 10^{-5} , используя метод наискорейшего спуска и метод градиентного спуска с дроблением шага.

2. Листинг

Листинг 1. Функции градиента и метода наискорейшего спуска

```
1      #include <functional>
2      #include <vector>
3      #include <fstream>
4      #include "../matrix_operations.h"
5      #include "../Lab_1_(One-dimensional_optimization)/
        golden_section_search.h"
6
7      int func_call_count{0};
8
9      template <typename T>
10     std::vector<T> gradient(const std::function<T(const std::
        vector<T> &)> &func, const std::vector<T> &point, T h =
        1e-6){
11         std::vector<T> grad(point.size());
12         for (size_t i = 0; i < point.size(); ++i){
13             std::vector<T> point_forward = point;
14             std::vector<T> point_backward = point;
15             point_forward[i] += h;
16             point_backward[i] -= h;
17             grad[i] = (func(point_forward) - func(
                point_backward)) / (2 * h);
18             func_call_count+=2;
```

```
19         }
20         return grad;
21     }
22
23     template <typename T>
24     std::pair<std::vector<T>, T> steepest_descent(const std::
25         function<T(const std::vector<T> &)> &func, std::vector<T
26         > initial_point, T tolerance = 1e-6, int max_iter =
27         1000, bool show_iterations=false) {
28         T alpha{1};
29         func_call_count = 0;
30         int iteration_count{0};
31         int gradient_count{0};
32         for (int iter = 0; iter < max_iter; ++iter){
33             auto grad = gradient(func, initial_point, tolerance
34             );
35             gradient_count++;
36             T grad_norm = compute2Norm(grad);
37             if (grad_norm < tolerance) {break;}
38             for (size_t i = 0; i < initial_point.size(); ++i){
39                 auto alpha_function = [&grad, &initial_point, &
40                 func](const T alpha) {
41                     func_call_count++;
42                     return func(initial_point - alpha*grad);
43                 };
44                 alpha = golden_section_search<T>(alpha_function
45                 , -1, 1, tolerance).first;
46                 if (alpha < tolerance){alpha = tolerance;}
47                 initial_point[i] -= alpha * grad[i];
48             }
49             iteration_count++;
50         }
51         func_call_count++;
52         if (show_iterations){
53             std::cout << "Convergence after " <<
54                 iteration_count << " iterations, " <<
55                 func_call_count << " function calls and " <<
56                 gradient_count << " gradient calculations" <<
```

```
std::endl;
48     }
49     return {initial_point, func(initial_point)};
50 }
```

Листинг 2. Функция метода градиентного спуска с дроблением шага

```
1  #include <functional>
2  #include <vector>
3  #include <fstream>
4  #include "../matrix_operations.h"
5  #include "steepest_descent.h"
6
7  template <typename T>
8  std::pair<std::vector<T>, T> gradient_descent_step_split(const
    std::function<T(const std::vector<T> &)> &func, std::vector<
    T> initial_point, T tolerance = 1e-6, T gamma = 0.5, T omega
    = 1, int max_iter = 1000, bool show_iterations=false) {
9      T alpha{1};
10     int iteration_count{0};
11     int gradient_count{0};
12     func_call_count = 0;
13     auto prev = initial_point;
14     for (int iter = 0; iter < max_iter; ++iter){
15         auto grad = gradient(func, initial_point, tolerance);
16         gradient_count++;
17         T grad_norm = compute2Norm(grad);
18         func_call_count+=2;
19         initial_point = initial_point - alpha * grad;
20         iteration_count++;
21         while (func(initial_point) > func(prev) - omega * alpha
            * grad_norm * grad_norm)
22         {
23             initial_point = prev;
24             alpha *= gamma;
25             initial_point = initial_point - alpha * grad;
26             func_call_count+=2;
27         }
28         if (compute2Norm(initial_point - prev ) < tolerance){
            break;}
    }
```

```
29         prev = initial_point;
30     }
31     func_call_count++;
32     if (show_iterations){
33         std::cout << "Convergence_after_" << iteration_count <<
            "iterations," << func_call_count << "function_
            and_" << gradient_count << "gradient_calculations"
            << std::endl;
34     }
35     return {initial_point, func(initial_point)};
36 }
```

3. Визуализация работы алгоритмов

Рассмотрим функцию

$$f(x) = 5x^2 - 4xy + 8y^2 + 8\sqrt{7}(2x + y) - 44. \quad (1)$$

Рассмотрим функцию Розенброка

$$J(x) = \alpha(x^2 - y)^2 + (x - 1)^2. \quad (2)$$

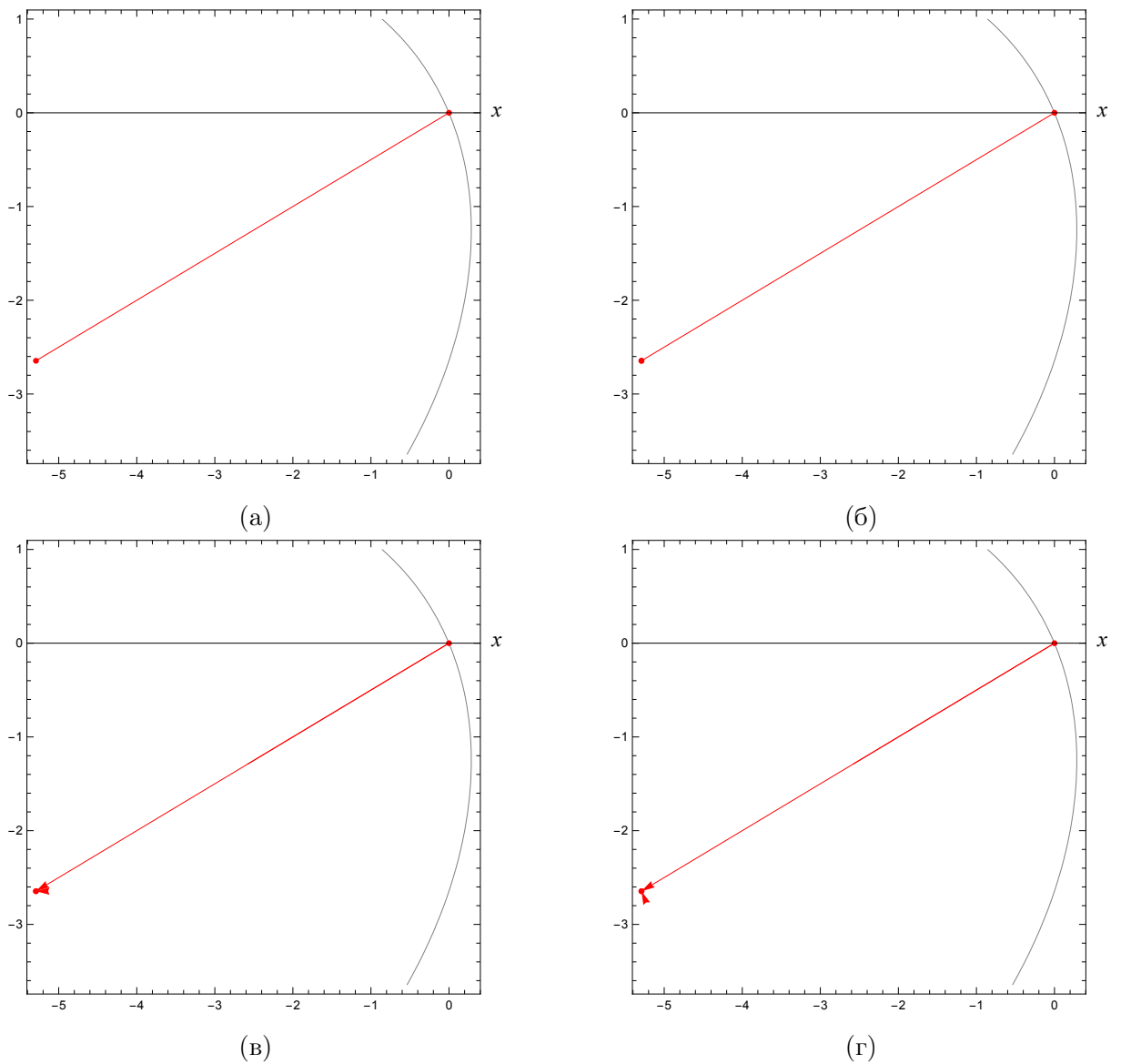


Рис. 1. Визуализация методов для функции (1) с начальным приближением $(0, 0)$

- (а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,
- (б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,
- (в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,
- (г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$

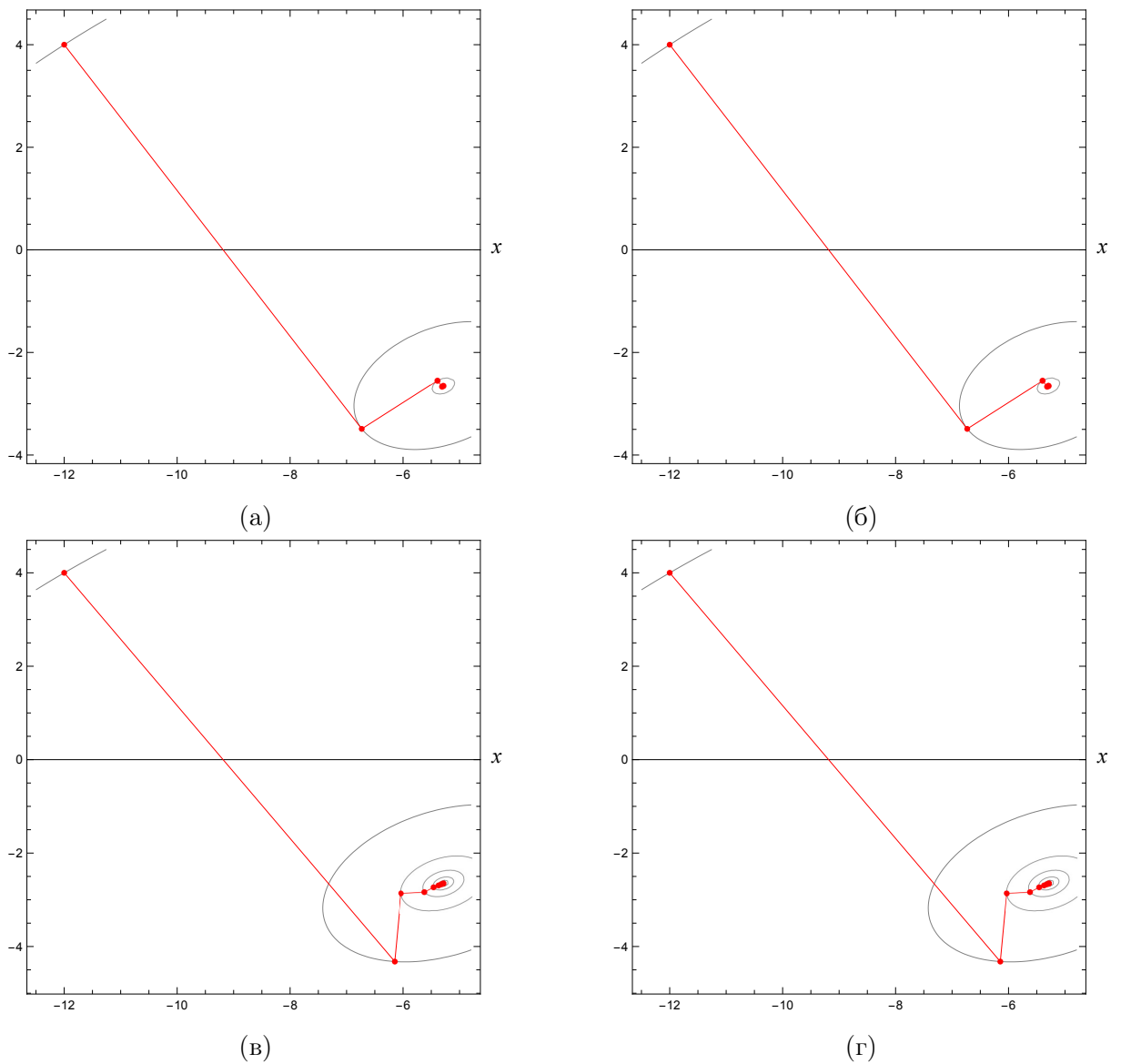
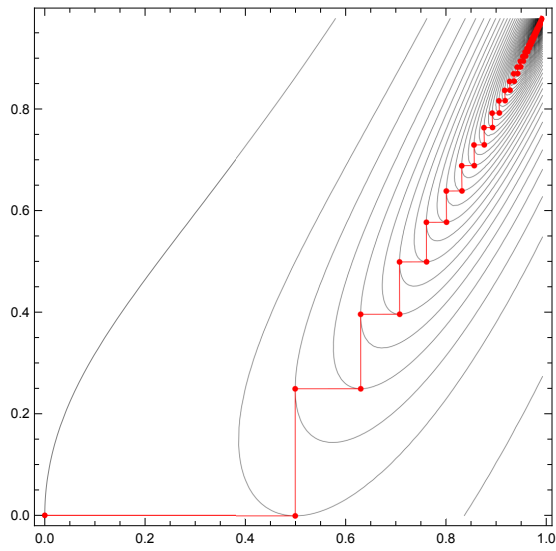
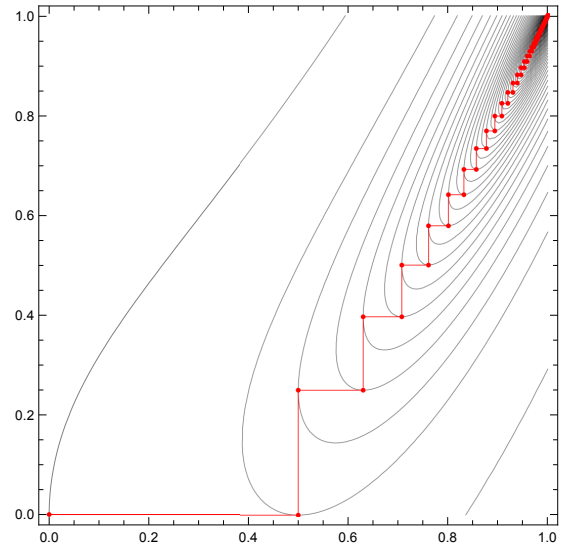


Рис. 2. Визуализация методов для функции (1) с начальным приближением $(-12, 4)$

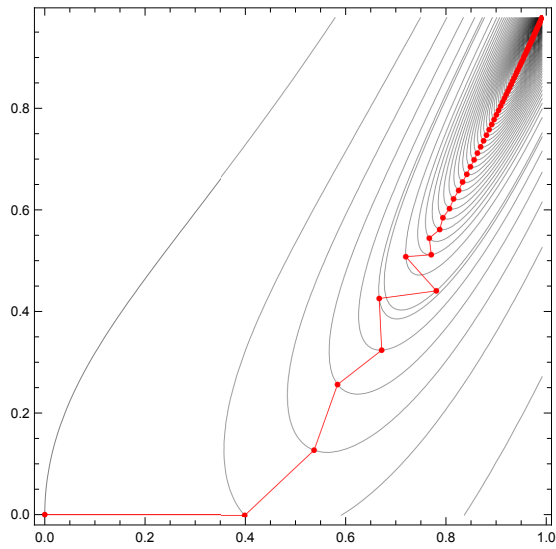
- (а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,
- (б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,
- (в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,
- (г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$



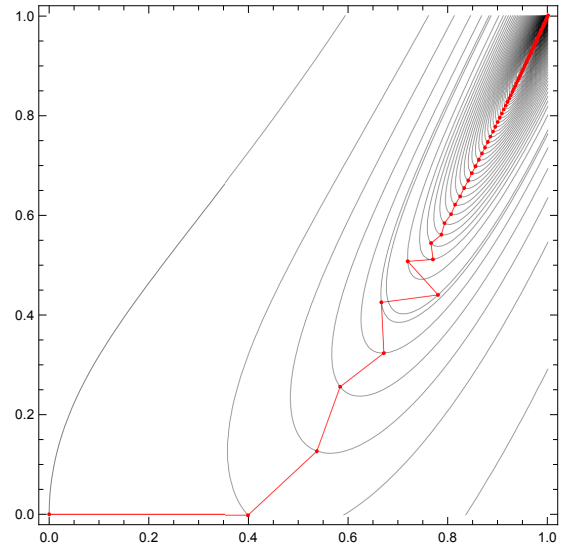
(а)



(б)



(в)



(г)

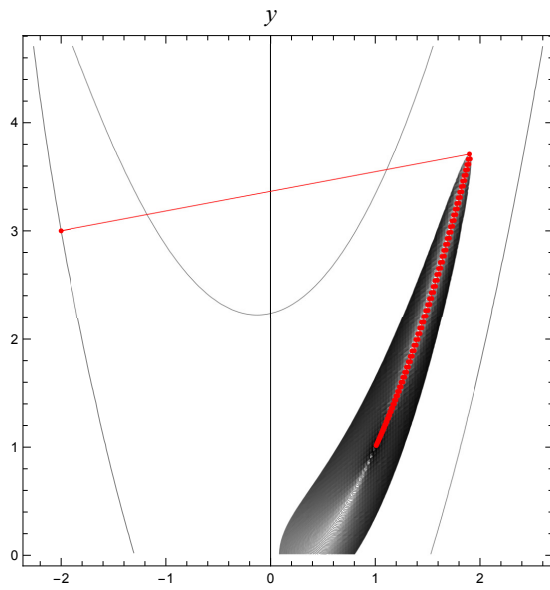
Рис. 3. Визуализация методов для функции Розенброка (2) при $\alpha = 2$ с начальным приближением $(0, 0)$

(а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,

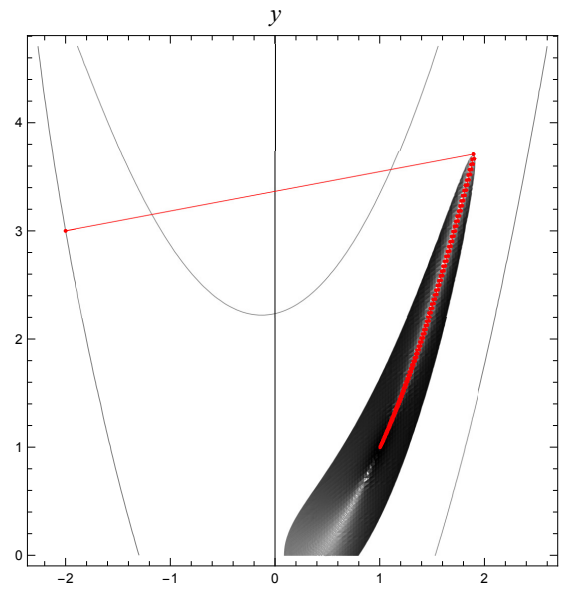
(б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,

(в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,

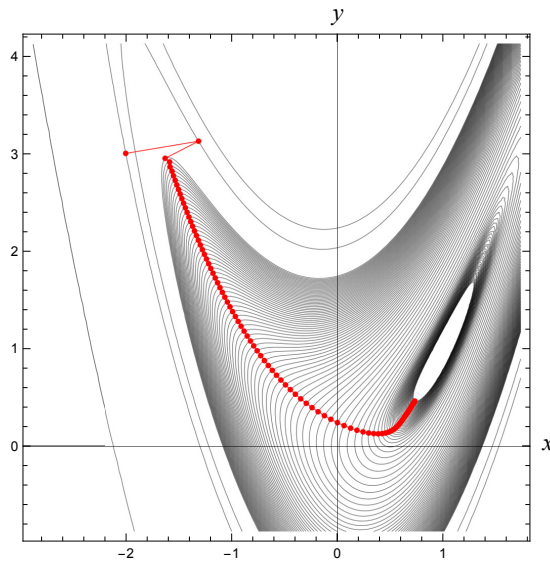
(г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$



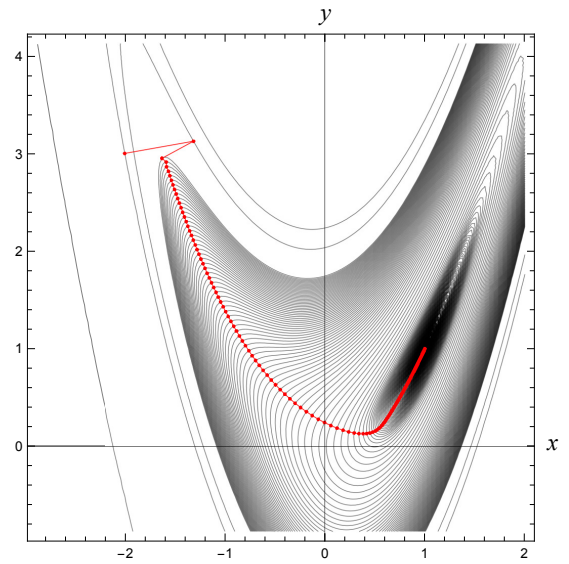
(а)



(б)



(в)



(г)

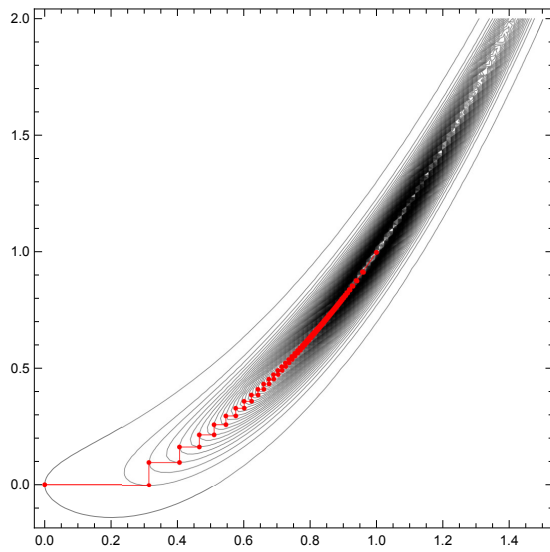
Рис. 4. Визуализация методов для функции Розенброка (2) при $\alpha = 2$ с начальным приближением $(-2, 3)$

(а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,

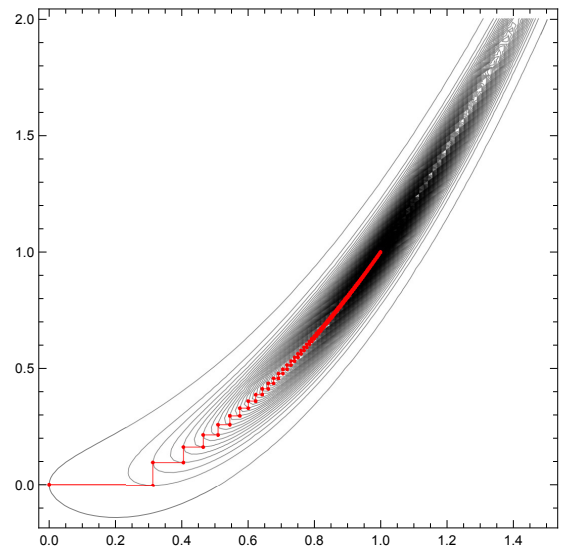
(б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,

(в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,

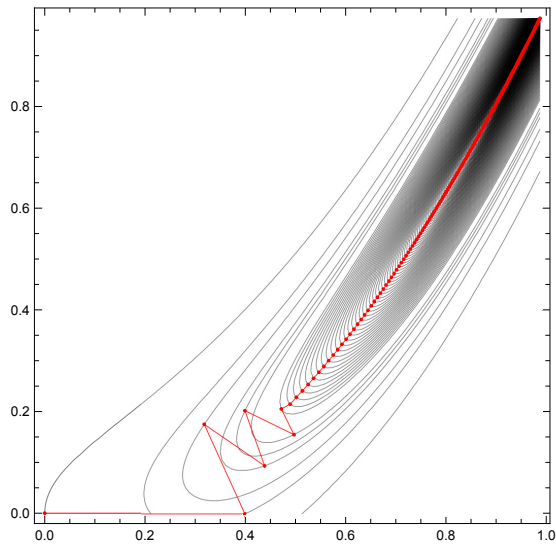
(г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$



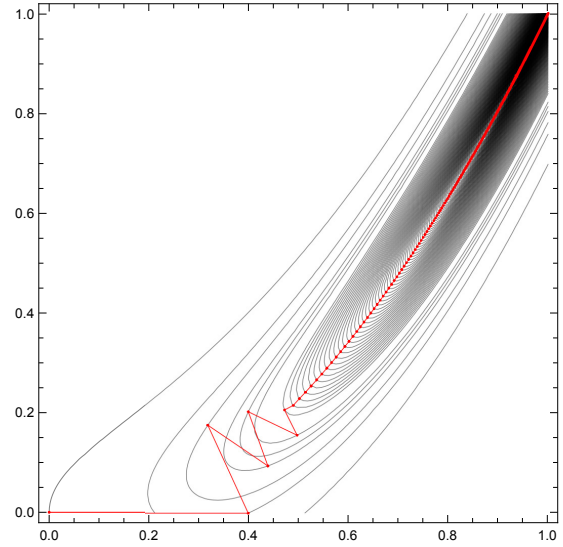
(а)



(б)



(в)



(г)

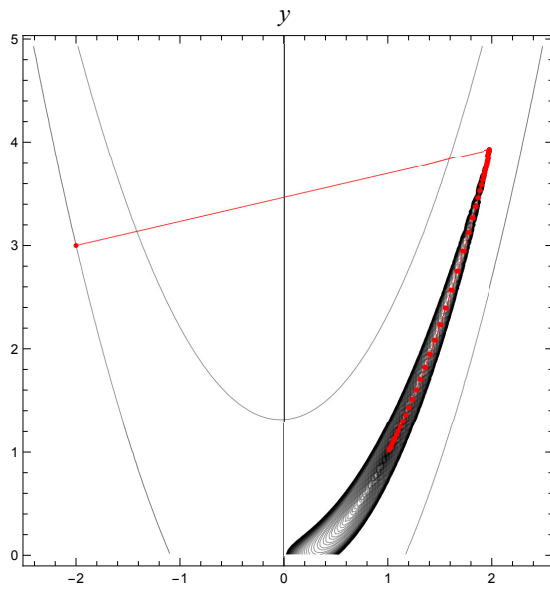
Рис. 5. Визуализация методов для функции Розенброка (2) при $\alpha = 11$ с начальным приближением $(0, 0)$

(а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,

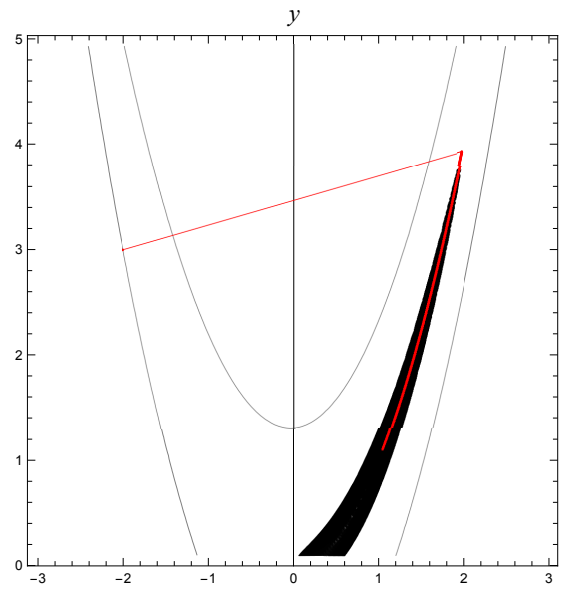
(б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,

(в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,

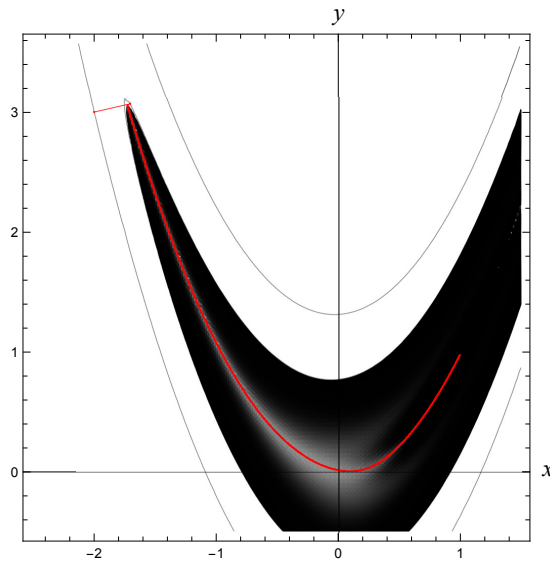
(г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$



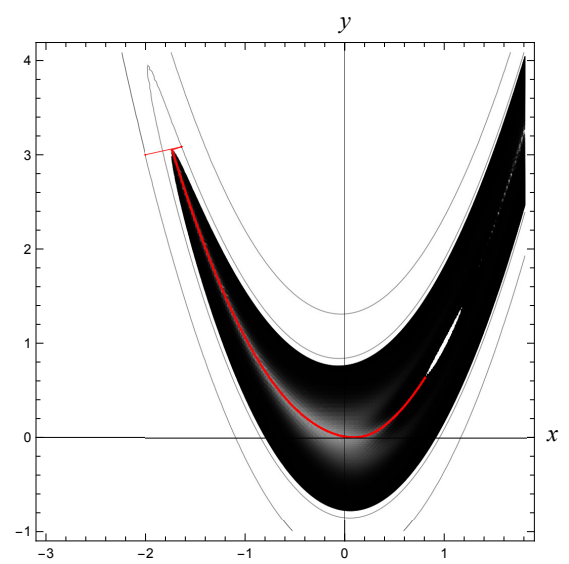
(а)



(б)



(в)



(г)

Рис. 6. Визуализация методов для функции Розенброка (2) при $\alpha = 11$ с начальным приближением $(-2, 3)$

(а) Метод наискорейшего спуска с $\varepsilon = 10^{-2}$,

(б) Метод наискорейшего спуска с $\varepsilon = 10^{-5}$,

(в) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-2}$,

(г) Метод градиентного спуска с дроблением шага с $\varepsilon = 10^{-5}$

4. Результаты работы алгоритмов

Таблица 1. Демонстрация работы методов для функции (1) при разных значениях параметра точности и различных начальных приближениях

Метод	(x_0, y_0)	ε	Кол-во итер.	Число вызовов функции	Число вычислений градиента	Точка минимума	Минимум
Метод наискор. спуска	(0, 0)	10^{-2}	1	53	2	(-5.29, -2.65)	-184.00
Метод наискор. спуска	(0, 0)	10^{-5}	1	53	2	(-5.29150, -2.64575)	-184.00000
Метод град. спуска	(0, 0)	10^{-2}	18	28	19	(-5.29, -2.65)	-184.00
Метод град. спуска	(0, 0)	10^{-5}	28	173	28	(-5.29150, -2.64575)	-184.00000
Метод наискор. спуска	(-12, 4)	10^{-2}	5	245	6	(-5.29, -2.65)	-184.00
Метод наискор. спуска	(-12, 4)	10^{-5}	8	389	9	(-5.29150, -2.64575)	-184.00000
Метод град. спуска	(-12, 4)	10^{-2}	11	69	11	(-5.29, -2.65)	-184.00
Метод град. спуска	(-12, 4)	10^{-5}	18	111	18	(-5.29151, -2.64575)	-184.00000

Таблица 2. Демонстрация работы методов для функции Розенброка с $\alpha = 2$ при разных значениях параметра точности и различных начальных приближениях

Метод	(x_0, y_0)	ε	Кол-во итер.	Число вызовов функции	Число вычислений градиента	Точка минимума	Минимум
Метод наискор. спуска	(0, 0)	10^{-2}	68	2453	69	(0.99, 0.98)	0.00
Метод наискор. спуска	(0, 0)	10^{-5}	173	6228	174	(0.99991, 0.99998)	0.00000
Метод град. спуска	(0, 0)	10^{-2}	84	507	84	(0.91, 0.81)	0.01
Метод град. спуска	(0, 0)	10^{-5}	438	2633	438	(0.99982, 0.99961)	0.00000
Метод наискор. спуска	(-2, 3)	10^{-2}	125	4505	126	(1.01, 1.02)	0.00
Метод наискор. спуска	(-2, 3)	10^{-5}	383	13793	384	(1.00001, 1.00002)	0.00001
Метод град. спуска	(-2, 3)	10^{-2}	429	593	429	(0.99, 0.98)	0.00
Метод град. спуска	(-2, 3)	10^{-5}	1170	7027	1170	(0.99964, 0.99922)	0.00000

Таблица 3. Демонстрация работы методов для функции Розенброка с $\alpha = 11$ при разных значениях параметра точности и различных начальных приближениях

Метод	(x_0, y_0)	ε	Кол-во итер.	Число вызовов функции	Число вычислений градиента	Точка минимума	Минимум
Метод наискор. спуска	(0, 0)	10^{-2}	139	5009	140	(0.99, 0.97)	0.00
Метод наискор. спуска	(0, 0)	10^{-5}	903	32513	904	(0.99999, 0.99998)	0.00000
Метод град. спуска	(0, 0)	10^{-2}	479	2883	479	(0.99, 0.97)	0.00
Метод град. спуска	(0, 0)	10^{-5}	1887	11331	1887	(0.99929, 0.99856)	0.00000
Метод наискор. спуска	(-2, 3)	10^{-2}	99	3569	100	(1.01, 1.02)	0.00
Метод наискор. спуска	(-2, 3)	10^{-5}	2179	78449	2180	(1.00001, 1.00002)	0.00000
Метод град. спуска	(-2, 3)	10^{-2}	3342	20065	3342	(0.99, 0.97)	0.00
Метод град. спуска	(-2, 3)	10^{-5}	8937	53635	8937	(0.99715, 0.99420)	0.00001

5. Выводы

Результаты, полученные в ходе лабораторной работы, позволяют провести анализ поведения методов градиентного спуска и наискорейшего спуска в различных условиях, а также проанализировать "стоимость" их использования в зависимости от параметра точности ε , начального приближения, выпуклости функции, а также ее овражности.

Анализ данных в (табл. 1) показывает, что для выпуклой квадратичной функции увеличение параметра точности ε ведет к росту числа итераций и вызовов функции. В частности, для метода наискорейшего спуска это увеличение составляет примерно в два раза, а для функции Розенброка рост числа вызовов функции при уменьшении ε оказывается еще более значительным (до 10-ти раз) (см. табл. 2 и 3).

Выбор начальной точки играет критически важную роль в сходимости метода для невыпуклых функций. Как видно из таблицы 1, в случае выпуклой квадратичной функции начальная точка практически не влияет на сходимость алгоритмов, однако может увеличивать число вычислений функции. Согласно данным (табл. 3)

для метода градиентного спуска с дроблением шага при $\varepsilon = 10^{-5}$ в случае смены начальной точки с $(0, 0)$ на $(5, 5)$ для функции Розенброка с параметром $\alpha = 11$ происходит значительный рост числа вычислений функции (более чем в 20 раз). Также, для метода градиентного спуска с $\varepsilon = 10^{-2}$ наблюдаются отличающиеся от ожидаемых результаты (например, $(-1.72, 3.05)$), а корректное приближение достигается только при более строгой точности.

Анализируя графики и таблицы приведенные в работе видно, что переход от выпуклой (квадратичной) функции к функции Розенброка приводит к ухудшению сходимости, росту вычислительной нагрузки и большей чувствительности алгоритмов к выбору начальной точки.

Сравнивая данные в (табл. 2) и (табл. 3) видно, что увеличение овражности существенно осложняет задачу оптимизации. В частности, для точки $(-2, 3)$ и $\varepsilon = 10^{-5}$ стоимость растет с 13793 ($\alpha = 2$) до 78449 ($\alpha = 11$) с 7027 ($\alpha = 2$) до 53635 ($\alpha = 11$) для методов наискорейшего и градиентного спуска соответственно.

Таким образом, для выпуклых квадратичных функций и функций с овражностью (например, функции Розенброка с $\alpha = 2$) более эффективным является метод градиентного спуска с дроблением, поскольку он требует меньше вычислений. Однако, иногда может быть предпочтительнее метод наискорейшего спуска, так как он демонстрирует большую устойчивость к расходимости и меньшую чувствительность к выбору начальной точки.